

# Day 31 – Dockerfile: Build Your Own Images

---

## Task 1: Your First Dockerfile

### Step 1: Create Project Folder

```
None
mkdir my-first-image
cd my-first-image
```

### Step 2: Create Dockerfile

```
None
nano Dockerfile
```

Paste:

```
None
FROM ubuntu

RUN apt-get update && apt-get install -y curl

CMD ["echo", "Hello from my custom image!"]
```

**My-first-image project1**

FROM ubuntu

WORKDIR /app

RUN apt-get update && apt-get install -y curl

CMD ["echo", "Hello from my custom image!"]

Save and exit.

---

## Step 3: Build Image

None

```
docker build -t my-ubuntu:v1 .
```

Explanation:

- `docker build` → build image
- `-t` → tag image
- `my-ubuntu:v1` → image name and version
- `.` → current directory (build context)

```

ubuntu@ip-172-31-81-141:~/my-first-image$ docker build -t my-first-image .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.048kB
Step 1/4 : FROM ubuntu
latest: Pulling from library/ubuntu
1c24335ddd46: Pulling fs layer
6f5c5aa4e145: Pulling fs layer
1c24335ddd46: Download complete
9bcf140d7f0f: Download complete
6f5c5aa4e145: Download complete
1c24335ddd46: Pull complete
6f5c5aa4e145: Pull complete
Digest: sha256:f3d28607ddd78734bb7f71f117f3c6706c666b8b76cbff7c9ff6e5718d46ff64
Status: Downloaded newer image for ubuntu:latest
--> f3d28607ddd7
Step 2/4 : WORKDIR /app
--> Running in 3ba8ccad4f53
--> Removed intermediate container 3ba8ccad4f53
--> fd4c92e8489e
Step 3/4 : RUN apt-get update && apt-get install -y curl
--> Running in 967f57f02454
Get:1 http://archive.ubuntu.com/ubuntu resolute InRelease [136 kB]
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [250 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [260 kB]
Get:5 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [142 kB]
Get:6 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:7 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [136 kB]
Get:8 http://archive.ubuntu.com/ubuntu resolute/main amd64 Packages [1874 kB]
Get:9 http://archive.ubuntu.com/ubuntu resolute/restricted amd64 Packages [189 kB]
Get:10 http://archive.ubuntu.com/ubuntu resolute/universe amd64 Packages [20.1 MB]
Get:11 http://archive.ubuntu.com/ubuntu resolute/multiverse amd64 Packages [352 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse amd64 Packages [3584 B]
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [160 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [279 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [250 kB]
Fetched 24.4 MB in 2s (14.5 MB/s)
Reading package lists...
Reading package lists...
Building dependency tree...

```

## Step 4: Verify Image

None

docker images

```

ubuntu@ip-172-31-81-141:~/my-first-image$ docker images

```

| IMAGE                 | ID           | DISK USAGE | CONTENT SIZE | EXTRA |
|-----------------------|--------------|------------|--------------|-------|
| devboard:latest       | c1c4554d2bb5 | 538MB      | 103MB        | U     |
| flaskapp:latest       | 1687f15e61fe | 1.64GB     | 421MB        | U     |
| my-first-image:latest | f4b69f4e237d | 254MB      | 77.1MB       | U     |
| node:20-alpine        | fb4cd12c85ee | 194MB      | 48.8MB       |       |
| python:3.14           | 6f473f84b09f | 1.63GB     | 432MB        |       |
| ubuntu:latest         | f3d28607ddd7 | 160MB      | 45.3MB       |       |

```

ubuntu@ip-172-31-81-141:~/my-first-image$

```

Output:

None

| REPOSITORY | TAG |
|------------|-----|
| my-ubuntu  | v1  |

---

## Step 5: Run Container

None

```
docker run my-ubuntu:v1
```

Output:

None

```
Hello from my custom image!
```

✓ Task 1 Complete

```
ubuntu@ip-172-31-81-141:~/my-first-image$ docker run my-first-image
Hello from my custom image!
ubuntu@ip-172-31-81-141:~/my-first-image$
```

---

## Task 2: Dockerfile Instructions

-----chatgpt-----

If you want to host an `index.html` page using Nginx in Docker, create these two files:

### Dockerfile

None

```
FROM nginx:alpine
```

```
COPY index.html /usr/share/nginx/html/index.html
```

EXPOSE 80

```
CMD ["nginx", "-g", "daemon off;"]
```

---

## index.html

None

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>My Docker Nginx Website</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin-top: 100px;
      background-color: #f4f4f4;
    }
    .container {
      background: white;
      padding: 30px;
      width: 60%;
      margin: auto;
      border-radius: 10px;
      box-shadow: 0 0 10px gray;
    }
    h1 {
      color: #0078d7;
    }
  </style>
</head>
<body>
```

```
<div class="container">
  <h1>Welcome to Docker + Nginx!</h1>
  <p>This website is running inside a Docker container.</p>
  <p>Build Date: 17 June 2026</p>
  <button onclick="showMessage()">Click Me</button>
  <p id="message"></p>
</div>

<script>
  function showMessage() {
    document.getElementById("message").innerHTML =
      "Congratulations! Your Nginx container is serving
this page.";
  }
</script>
</body>
</html>
```

---

## Build the image

None

```
docker build -t my-nginx-site .
```

## Run the container

None

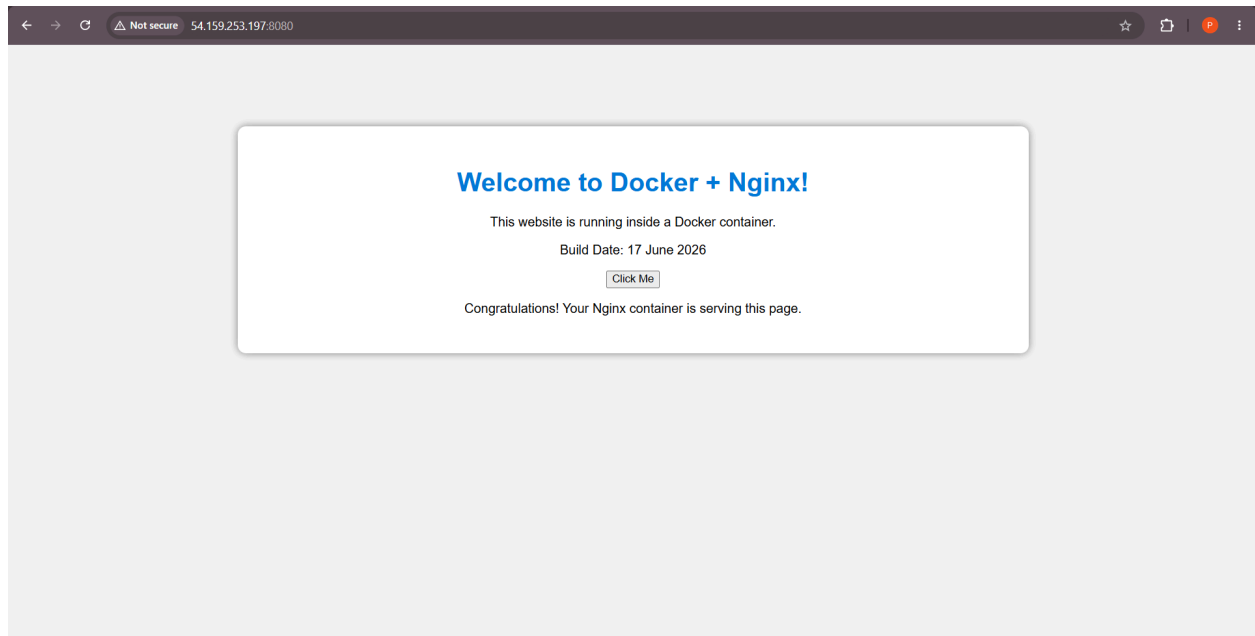
```
docker run -d -p 8080:80 --name nginx-web my-nginx-site
```

## Access the website

Open:

None

```
http://localhost:8080
```



You should see your custom HTML page instead of the default Nginx welcome page.

### **Flask-app-ecs-project2-----Dockerfile**

FROM python:3.14

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

CMD ["python", "[run.py](#)"]

### **Index.html project3-----Dockerfile**

FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

```
ubuntu@ip-172-31-81-141:~/indexexample$ ls
Dockerfile  index.html
ubuntu@ip-172-31-81-141:~/indexexample$ cat Dockerfile
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

---

## Meaning of Each Instruction

### FROM

Base image.

None

**FROM** nginx:alpine

### RUN

Runs commands while building image.

None

**RUN** echo "Build Successful"

### COPY

Copies files from host to image.

None

**COPY** index.html .



---

## WORKDIR

Sets working directory.

None

```
WORKDIR /usr/share/nginx/html
```

---

## EXPOSE

Documents port.

None

```
EXPOSE 80
```

---

## CMD

Default command.

None

```
CMD ["nginx","-g","daemon off;"]
```

---

# Task 3: CMD vs ENTRYPOINT

---

## CMD Example

Dockerfile:

None

```
FROM ubuntu
```

```
CMD ["echo", "hello"]
```

Build:

```
None  
docker build -t cmd-demo .
```

Run:

```
None  
docker run cmd-demo
```

Output:

```
None  
hello
```

---

Override CMD:

```
None  
docker run cmd-demo date
```

Output:

```
None  
Sun Jun 15 ...
```

CMD gets replaced.

---

## ENTRYPOINT Example

Dockerfile:

None

```
FROM ubuntu
```

```
ENTRYPOINT ["echo"]
```

Build:

None

```
docker build -t entry-demo .
```

Run:

None

```
docker run entry-demo hello
```

Output:

None

```
hello
```

---

Run again:

None

```
docker run entry-demo docker
```

Output:

None

```
docker
```

Arguments are appended.

---

## CMD vs ENTRYPOINT

| CMD                        | ENTRYPOINT                    |
|----------------------------|-------------------------------|
| Default command            | Fixed command                 |
| Easily overridden          | Not overridden easily         |
| Used for optional defaults | Used for mandatory executable |

## Use CMD

None

```
CMD ["python", "app.py"]
```

When user may want another command.

---

## Use ENTRYPOINT

None

```
ENTRYPOINT ["python"]
```

When image should always run Python.

---

# Task 4: Build a Simple Web App

---

## Step 1: Create Folder

None

```
mkdir my-website  
cd my-website
```

---

## Step 2: Create HTML

None

```
nano index.html
```

Paste:

None

```
<!DOCTYPE html>
<html>
<head>
<title>Docker Website</title>
</head>
<body>
<h1>Hello From Docker!</h1>
</body>
</html>
```

---

## Step 3: Create Dockerfile

None

```
FROM nginx:alpine
```

```
COPY index.html /usr/share/nginx/html/index.html
```

```
EXPOSE 80
```

---

## Step 4: Build

None

```
docker build -t my-website:v1 .
```

---

## Step 5: Run

None

```
docker run -d -p 8080:80 --name website my-website:v1
```

---

## Step 6: Verify

Open:

None

```
http://localhost:8080
```

or

None

```
http://EC2-PUBLIC-IP:8080
```

You should see:

None

```
Hello From Docker!
```

☒ Task 4 Complete

---

## Task 5: .dockerignore

Create file:

None

```
nano .dockerignore
```

Paste:

```
None
node_modules
.git
*.md
.env
```

---

## Verify

Build image:

```
None
docker build -t ignore-demo .
```

Check build context size:

```
None
Sending build context to Docker daemon
```

Size becomes smaller because ignored files are excluded.

---

## Task 6: Build Optimization

---

### Example Dockerfile

```
None
FROM ubuntu

COPY . .

RUN apt-get update
```

```
CMD ["bash"]
```

Build:

None

```
docker build -t cache-demo .
```

---

Modify:

None

```
README.md
```

Build again:

None

```
docker build -t cache-demo .
```

Docker rebuilds from COPY layer onward.

---

## Better Dockerfile

None

```
FROM ubuntu
```

```
RUN apt-get update
```

```
COPY . .
```

```
CMD ["bash"]
```

Now package installation layer is cached.

Only COPY layer rebuilds.



---

## Why Layer Order Matters

Docker builds images in layers:

None

Layer 1 → FROM

Layer 2 → RUN

Layer 3 → COPY

Layer 4 → CMD

If Layer 3 changes:

None

Layer 1 = Cached

Layer 2 = Cached

Layer 3 = Rebuilt

Layer 4 = Rebuilt

Best Practice:

None

Put stable commands first

Put frequently changing files last

This makes builds much faster.

---

## Important Commands Cheat Sheet

None

`docker build -t image:tag .`

`docker images`

```
docker run image:tag
```

```
docker run -d -p 8080:80 image:tag
```

```
docker ps
```

```
docker ps -a
```

```
docker stop container_name
```

```
docker rm container_name
```

```
docker rmi image_name
```

```
docker logs container_name
```

## **Devboard project4-----Dockerfile**

From node:20-alpine

WORKDIR /app

COPY package\*.json ./

RUN npm ci

COPY . .

RUN npm run build

COPY . .

EXPOSE 5173

CMD ["npm","run","preview","--","--host","0.0.0.0","--port","5173"]

This is enough to complete **Day 31 Dockerfile Challenge** and create your [day-31-dockerfile.md](#) for GitHub.

If you want to delete **all containers** and **all images**, run these commands in order.

### 1. Stop all running containers

None

```
docker stop $(docker ps -q)
```

### 2. Delete all containers

None

```
docker rm $(docker ps -aq)
```

### 3. Delete all images

None

```
docker rmi $(docker images -q)
```

If Docker says an image is being used by a container, make sure you've removed all containers first (steps 1 and 2).

### 4. Verify everything is deleted

None

```
docker ps -a  
docker images
```

Both should show no containers/images.

